README for C++ Concurrency Project

https://gemini.google.com/app/5adc152474872d12

User prompt: Help me create a good readme for my school project

Response: This project hits hard—using low-level C++ concurrency and data structures as narrative metaphors for executive dysfunction is brilliant.

Here is a ready-to-use README structure designed for markdown:

```
# [UN]FOCUSED
```

~~~
**[UN]FOCUSED** is a interactive C++ code poem that models the daily internal mechanics of living with ADHD[cite: 2]. By mapping core computer science concepts—concurrency, context switches, stack allocation, and memory disorder—to human executive dysfunction, the program translates abstract psychological struggles into tangible, terminal-based execution behavior[cite: 1,

Where the outside world sees laziness or distraction, this project uses code to demonstrate the constant, exhausting background friction of a brain fighting to function[cite: 2].

---

**Concept & Technical Metaphors**

The codebase leverages modern C++ mechanics as literal narrative devices[cite: 1]:

*   **Asynchronous Thought Collisions (`std::thread`, `std::atomic`):** Task execution runs on a simulated loop that is continuously context-switched and interrupted mid-execution by asynchronous, intrusive thoughts[cite: 1].
*   **Terminal Regressions (`\r\033[K`):** Task progress in the terminal is actively overwritten and reset as focus breaks, causing execution indices to loop backward[cite: 1].
*   **Stack Allocation as Emotional Capacity (`std::stack`):** Unprocessed frustrations, self-doubt, and sensory friction are pushed sequentially onto a stack until memory budget limits are reached[cite: 1].
*   **Order Instability (`std::unordered_map`):** Hash table traversal reflects the non-sequential, highly chaotic nature of fragmented context switching[cite: 1].
*   **Executive Burnout (`quick_exit`):** Rather than a graceful thread termination, the system experiences a critical executive function exhaustion, forcing an immediate early exit[cite: 1].

---

**Build & Execution**

**Prerequisites**
*   C++17 compliant compiler (e.g., `g++` or `clang++`)[cite: 3]
*   `make` utility[cite: 3]
*   An ANSI-compatible terminal (for screen-clearing and escape sequence effects)[cite: 1]

**Quick Start**

1. Clone or download the repository.
2. Build and run immediately using the included Makefile:
   ```bash
   make run
~~~

Manual Compilation

If you prefer to compile manually:

```
g++ -std=c++17 -Wall -Wextra -O2 UNFOCUSED_AI_REFACTOR.cpp -o UNFOCUSED_AI_REFACTOR
./UNFOCUSED_AI_REFACTOR
```

Clean Build Artifacts

```
make clean
```

User prompt: the ascii art looks horrible in the re adme

Response: Fair point—raw ASCII art often breaks terribly on mobile screens or standard GitHub markdown renders because line wrapping distorts the character alignment.

Here are the three best ways to fix it depending on the look you want:

Option 1: Wrap in Code Blocks (Recommended)

Enclose the ASCII art inside triple backticks (```). This forces markdown to render it in a fixed-width monospace font without line-wrapping.

Option 2: Use Scaled ANSI Block Art (Cleaner & Shorter)

Large block fonts struggle on narrow screens. A compact ANSI layout scales significantly better across device displays:

~~~
```text
  _   _ _  _ _____ ___   ___ _  _ ___ ___
 | | | | \| |  ___/ _ \ / __| | | / __| __|
 | |_| | .` | _| | (_) | (__| |_| \__ \ _|
  \___/|_|\_|_|   \___/ \___|\___/|___/___|


---

**Option 3: HTML `<pre>` Tags**

If GitHub's code block background color ruins the artistic aesthetic, wrap it in `<pre>` tags instead to preserve spacing without adding a grey syntax box:

```html
<pre>
~~~

```
</pre>
```